

CO5 – AT2: Pseudocode Writing Task (Algorithm Design)

Name:A.Rakesh

Reg no:192525388

Course: Database Management Systems (CSA0504)

Query Processing, Optimization & Transaction Management

Q1. Query Processing Pipeline (SQL Input → Parsing → Optimization → Execution → Output)

Pseudocode

ALGORITHM QueryProcessingPipeline(sql_query)

BEGIN

 // Step 1: Lexical and Syntax Analysis (Parsing)

 tokens = Lexer(sql_query)

 parse_tree = Parser(tokens) // builds parse tree, checks grammar

 IF parse_tree is INVALID THEN

 RETURN ERROR('Syntax Error')

 END IF

 // Step 2: Semantic Analysis

 IF NOT ValidateSchema(parse_tree) THEN // checks tables/columns exist,

 RETURN ERROR('Semantic Error') // type compatibility, privileges

 END IF

 // Step 3: Translate to internal (relational algebra) query tree

 query_tree = ConvertToRelationalAlgebra(parse_tree)

 // Step 4: Query Optimization

```

candidate_plans = GenerateEquivalentPlans(query_tree) // apply RA laws
FOR EACH plan IN candidate_plans DO
    plan.cost = EstimateCost(plan) // uses statistics: cardinality,
END FOR // selectivity, index availability
best_plan = SelectMinCostPlan(candidate_plans)

// Step 5: Code Generation
executable_plan = GenerateExecutionCode(best_plan)

// Step 6: Execution
result = ExecutionEngine.Run(executable_plan)

// Step 7: Output
RETURN FormatResult(result)
END

```

Q2. Cost-Based Query Optimization – Join Ordering (I/O Cost Comparison)

Pseudocode (Dynamic-Programming Join Order Selection)

ALGORITHM CostBasedJoinOrdering(Relations R1, R2, ..., Rn)

BEGIN

// BestPlan[S] stores the cheapest plan for relation subset S

FOR EACH Ri IN {R1,...,Rn} DO

BestPlan[{Ri}] = (accessPlan = SeqScanOrIndex(Ri),

cost = EstimateScanCost(Ri))

END FOR

FOR size = 2 TO n DO

FOR EACH subset S of {R1,...,Rn} WITH |S| = size DO

BestPlan[S].cost = INFINITY

```
FOR EACH split (S1, S2) WHERE S1 UNION S2 = S, S1 INTERSECT S2 =  
EMPTY DO
```

```
    left = BestPlan[S1]
```

```
    right = BestPlan[S2]
```

```
    joinCost = IOCost(left, right)    // e.g. left.cost + right.cost
```

```
                // + pages(left)*pages(right)/M
```

```
    totalCost = left.cost + right.cost + joinCost
```

```
    IF totalCost < BestPlan[S].cost THEN
```

```
        BestPlan[S] = (plan = Join(left.plan, right.plan),
```

```
                        cost = totalCost)
```

```
    END IF
```

```
END FOR
```

```
END FOR
```

```
END FOR
```

```
RETURN BestPlan[{R1,...,Rn}]    // optimal join order and estimated cost
```

```
END
```

```
FUNCTION IOCost(leftPlan, rightPlan)
```

```
BEGIN
```

```
    // Nested-loop style I/O cost estimate
```

```
    RETURN pages(leftPlan) + (pages(leftPlan) * pages(rightPlan)) / bufferPages(M)
```

```
END
```

Q3. Nested Loop Join Algorithm + Time Complexity Analysis

Pseudocode

ALGORITHM NestedLoopJoin(R, S, condition)

// R = outer relation, S = inner relation

// B(R), B(S) = number of blocks (pages); M = buffer pages available

BEGIN

result = EMPTY SET

FOR EACH block Br IN R DO // read one outer block at a time

FOR EACH block Bs IN S DO // scan inner relation fully

FOR EACH tuple r IN Br DO

FOR EACH tuple s IN Bs DO

IF condition(r, s) = TRUE THEN

result = result UNION {r JOIN s}

END IF

END FOR

END FOR

END FOR

END FOR

RETURN result

END

Time / I-O Complexity Analysis

Let B(R) and B(S) be the number of disk blocks (pages) of R and S, and T(R), T(S) be the number of tuples. With only 1 buffer page for the inner relation (simple nested loop, block-at-a-time on outer, tuple re-scan on inner):

Disk I/O cost = $B(R) + T(R) * B(S)$

(R is scanned once; for every tuple of R, S is re-scanned entirely)

Block Nested Loop (M buffer pages, one held for output, one for inner scan, M-2 for outer blocks per pass):

$$= B(R) + \text{ceil}(B(R) / (M - 2)) * B(S)$$

CPU time complexity = $O(T(R) * T(S))$ -- every tuple pair is compared

Best case occurs when the smaller relation fits entirely in the buffer ($B(S) \leq M - 2$), reducing cost to $B(R) + B(S)$, i.e., each relation is read only once.

Q4. Deadlock Detection Using Wait-For Graph (WFG) Cycle Detection

Pseudocode

ALGORITHM DetectDeadlock(WFG)

// WFG = directed graph; node = transaction T_i ; edge $T_i \rightarrow T_j$ means

// T_i is waiting for a lock held by T_j

BEGIN

FOR EACH node T IN WFG.nodes DO

visited[T] = FALSE

inStack[T] = FALSE

END FOR

FOR EACH node T IN WFG.nodes DO

IF visited[T] = FALSE THEN

IF HasCycleDFS(T, WFG, visited, inStack) = TRUE THEN

RETURN ('Deadlock Detected', GetCycle())

END IF

END IF

END FOR

RETURN 'No Deadlock'

END

```
FUNCTION HasCycleDFS(T, WFG, visited, inStack)
```

```
BEGIN
```

```
    visited[T] = TRUE
```

```
    inStack[T] = TRUE
```

```
    PushToPathStack(T)
```

```
    FOR EACH neighbor N IN WFG.adjacent(T) DO    // T waits-for N
```

```
        IF visited[N] = FALSE THEN
```

```
            IF HasCycleDFS(N, WFG, visited, inStack) = TRUE THEN
```

```
                RETURN TRUE
```

```
            END IF
```

```
        ELSE IF inStack[N] = TRUE THEN
```

```
            RETURN TRUE    // back-edge found -> cycle -> deadlock
```

```
        END IF
```

```
    END FOR
```

```
    inStack[T] = FALSE
```

```
    PopFromPathStack()
```

```
    RETURN FALSE
```

```
END
```

```
// On deadlock: choose a victim transaction (e.g., youngest / lowest cost)
```

```
// from the cycle and ROLLBACK it to break the cycle.
```

Q5. Basic Two-Phase Locking (2PL) Protocol

Pseudocode

ALGORITHM BasicTwoPhaseLocking(Transaction T)

BEGIN

 phase = 'GROWING'

 lockSet[T] = EMPTY

 FOR EACH operation OP IN T.operations DO

 IF OP requires READ on item X THEN

 REQUEST Shared_Lock(X, T)

 ELSE IF OP requires WRITE on item X THEN

 REQUEST Exclusive_Lock(X, T)

 END IF

 END FOR

END

FUNCTION RequestLock(item X, Transaction T, mode)

BEGIN

 IF phase[T] = 'SHRINKING' THEN

 ERROR('Cannot acquire new lock after releasing one - 2PL violated')

 END IF

 IF X is UNLOCKED THEN

 GRANT lock(X, T, mode)

 lockSet[T] = lockSet[T] UNION {X}

 ELSE IF X is locked in SHARED AND mode = SHARED THEN

 GRANT lock(X, T, mode) // compatible, add to lock set

 lockSet[T] = lockSet[T] UNION {X}

```

ELSE
    WAIT until lock(X) is released      // put T in wait queue for X
    // re-check compatibility once notified, then GRANT
END IF
END

FUNCTION ReleaseLock(item X, Transaction T)
BEGIN
    phase[T] = 'SHRINKING'      // once first unlock happens, no new locks
    UNLOCK(X, T)
    lockSet[T] = lockSet[T] - {X}
    WAKE UP waiting transactions for X, re-check compatibility
END

// Growing Phase : only acquire locks, never release
// Shrinking Phase: only release locks, never acquire (guarantees
//      conflict-serializable schedules)

```

Q6. Timestamp-Based Concurrency Control with Thomas' Write Rule

Pseudocode

```

// Each data item X maintains:
//  Read_TS(X) - largest timestamp of a transaction that read X
//  Write_TS(X) - largest timestamp of a transaction that wrote X
// Each transaction T is assigned TS(T) at start (e.g., system clock / counter)

```

```

ALGORITHM TimestampOrdering_ThomasWriteRule(Transaction T, item X, operation OP)
BEGIN
    IF OP = READ(X) THEN
        IF TS(T) < Write_TS(X) THEN

```



```

    ROLLBACK(T)          // T is reading a value already
    RETURN 'ABORTED'      // overwritten by a newer transaction
ELSE
    EXECUTE Read(X)
    Read_TS(X) = MAX(Read_TS(X), TS(T))
END IF

ELSE IF OP = WRITE(X) THEN
    IF TS(T) < Read_TS(X) THEN
        ROLLBACK(T)      // a younger transaction already read
        RETURN 'ABORTED'  // the old value; writing now is unsafe

    ELSE IF TS(T) < Write_TS(X) THEN
        // Thomas' Write Rule: this write is obsolete because a later
        // transaction has already written a newer value - IGNORE it
        // instead of aborting T (improves concurrency over basic TO)
        SKIP Write(X)
        RETURN 'IGNORED'

    ELSE
        EXECUTE Write(X)
        Write_TS(X) = TS(T)
    END IF
END IF

RETURN 'SUCCESS'

END

```

Q7. ARIES Recovery Algorithm (Analysis, Redo, Undo)

Pseudocode

ALGORITHM ARIES_Recovery(LogFile, LastCheckpoint)

BEGIN

// ----- PHASE 1: ANALYSIS -----

DirtyPageTable = LOAD from LastCheckpoint

TransactionTable = LOAD from LastCheckpoint

FOR EACH log record LR FROM LastCheckpoint TO END OF LOG DO

IF LR.type IN {UPDATE} THEN

IF LR.pageID NOT IN DirtyPageTable THEN

DirtyPageTable[LR.pageID] = LR.LSN // recLSN

END IF

TransactionTable[LR.transID].lastLSN = LR.LSN

TransactionTable[LR.transID].status = 'ACTIVE'

ELSE IF LR.type = COMMIT THEN

TransactionTable[LR.transID].status = 'COMMITTED'

ELSE IF LR.type = END THEN

REMOVE LR.transID FROM TransactionTable

END IF

END FOR

losers = { T IN TransactionTable | T.status != 'COMMITTED' }

// ----- PHASE 2: REDO -----

startLSN = MIN(recLSN) over all pages in DirtyPageTable

FOR EACH log record LR FROM startLSN TO END OF LOG DO

IF LR.type = UPDATE THEN

page = FETCH(LR.pageID)

IF LR.pageID IN DirtyPageTable

```

    AND LR.LSN >= DirtyPageTable[LR.pageID].recLSN
    AND page.LSN < LR.LSN THEN
        REDO(LR)           // reapply the change
        page.LSN = LR.LSN
    END IF
END IF
END FOR

// ----- PHASE 3: UNDO -----
ToUndo = priority queue of lastLSN for each T in losers (descending)
WHILE ToUndo is NOT EMPTY DO
    LSN = REMOVE MAX from ToUndo
    LR = READ log record at LSN
    IF LR.type = UPDATE THEN
        UNDO(LR)           // restore before-image
        WRITE Compensation Log Record (CLR) with undoNextLSN
        IF LR.prevLSN EXISTS THEN
            ADD LR.prevLSN TO ToUndo
        ELSE
            WRITE END log record for LR.transID
        END IF
    END IF
END IF
END WHILE

RETURN 'Database Recovered to Consistent State'
END

```

Q8. Shadow Paging Recovery

Pseudocode

ALGORITHM ShadowPaging(Transaction T)

BEGIN

ShadowPageTable = COPY OF CurrentPageTable // saved at T start, on stable storage

CurrentPageTable = COPY OF ShadowPageTable // working copy, used during T

FOR EACH write operation W(X) IN T DO

oldPage = LocatePage(CurrentPageTable, X)

newPage = ALLOCATE new disk block // never overwrite in place

COPY data FROM oldPage TO newPage

APPLY update to newPage

CurrentPageTable[X] = newPage // point to new page

// ShadowPageTable[X] still points to the untouched old page

END FOR

IF T reaches COMMIT THEN

WRITE CurrentPageTable to disk (new pages already durable)

ATOMICALLY: ShadowPageTable = CurrentPageTable // single pointer swap

DISCARD old shadow pages (garbage collect)

RETURN 'COMMITTED'

ELSE IF T FAILS or is ABORTED THEN

DISCARD CurrentPageTable and all new pages written

CurrentPageTable = ShadowPageTable // rollback: revert to shadow copy

RETURN 'ROLLED BACK'

END IF

END

```
// On system crash before commit: on restart, the on-disk root simply
// still points to ShadowPageTable, so the database is automatically
// consistent - no REDO/UNDO log processing is required.
```

Q9. Immediate Update Recovery (UNDO/REDO Log Processing)

Pseudocode

ALGORITHM ImmediateUpdateRecovery(LogFile)

BEGIN

 UndoList = EMPTY LIST // transactions with no COMMIT record

 RedoList = EMPTY LIST // transactions that committed

 // ---- Pass 1: scan log to classify transactions ----

 FOR EACH log record LR IN LogFile DO

 IF LR.type = START_TRANSACTION THEN

 ADD LR.transID TO UndoList

 ELSE IF LR.type = COMMIT THEN

 MOVE LR.transID FROM UndoList TO RedoList

 END IF

 END FOR

 // Transactions remaining in UndoList never committed -> must be undone

 // Transactions in RedoList committed -> must be redone (durability)

 // ---- Pass 2: UNDO phase (scan log backward) ----

 FOR EACH log record LR IN LogFile TRAVERSED IN REVERSE DO

 IF LR.type = UPDATE AND LR.transID IN UndoList THEN

 WRITE(LR.dataItem, LR.beforeImage) // restore old value

 END IF

 END FOR

```

// ---- Pass 3: REDO phase (scan log forward) ----

FOR EACH log record LR IN LogFile TRAVERSED FORWARD DO
    IF LR.type = UPDATE AND LR.transID IN RedoList THEN
        WRITE(LR.dataItem, LR.afterImage)  // reapply new value
    END IF
END FOR

WRITE Checkpoint record

RETURN 'Recovery Complete'

END

// Immediate Update: changes are written to disk as the transaction runs
// (before commit) -> UNDO is required for uncommitted transactions;
// a crash after commit but before all pages are flushed -> REDO required.

Q10. Database Connectivity Manager (Connection Pooling, Execution, Rollback)

Pseudocode

ALGORITHM DBConnectivityManager

BEGIN

    ConnectionPool = INITIALIZE pool with minSize idle connections

    MAX_POOL_SIZE = N

END

FUNCTION GetConnection()

BEGIN

    IF ConnectionPool.hasIdleConnection() THEN

        conn = ConnectionPool.removeIdleConnection()

    ELSE IF ConnectionPool.size() < MAX_POOL_SIZE THEN

```

```

        conn = CreateNewConnection(DB_URL, credentials)
        ConnectionPool.add(conn)
ELSE
    WAIT until a connection is released, OR timeout -> ERROR('Pool exhausted')
END IF

conn.status = 'IN_USE'

RETURN conn
END

```

```

FUNCTION ExecuteQuery(sql, params)
BEGIN
    conn = GetConnection()
    TRY
        conn.beginTransaction()
        statement = conn.prepareStatement(sql, params)
        result = statement.execute()
        conn.commit()

        RETURN result
    CATCH SQLException AS e
        conn.rollback()           // undo partial changes on failure
        LogError(e)
        RAISE e
    FINALLY
        ReleaseConnection(conn)   // always return to pool
    END TRY
END

```

```

FUNCTION ReleaseConnection(conn)

```

BEGIN

IF conn.isValid() THEN

conn.status = 'IDLE'

ConnectionPool.returnConnection(conn)

ELSE

ConnectionPool.discard(conn)

ConnectionPool.add(CreateNewConnection(DB_URL, credentials))

END IF

END